

Data Structures

Lecture 10

Sorting

References:

Data Structures and Algorithms in Java

Michael T. Goodrich and Roberto Tamassia.



University of Kufa
Faculty of Computer Science
and Mathematics

Dr. Munqath Al-Attar

Information Technology Research and
Development Center ITRDC

Email:

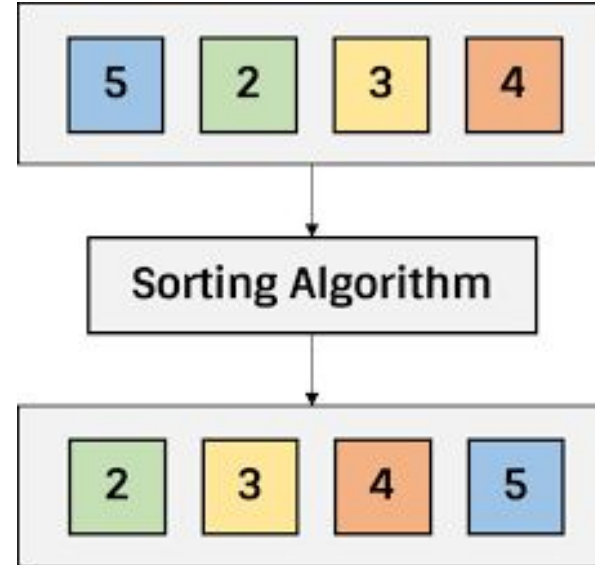
munqith.alattar@uokufa.edu.iq

Website:

<http://staff.uokufa.edu.iq/profile.html?munqith.alattar>

Topics

- **Sorting**
 - Bubble sort
 - Selection sort
 - Insertion sort
 - Shell sort
 - Quicksort
 - Heapsort



Data Structures

Sorting

- **Sorting:** arrange elements of a list into an order.
- Orders can be **numerical** order and **alphabetical** order, and either **ascending** or **descending**.
- Sorting is useful for other algorithms (such as search).
- For example, the contact list in a phone is sorted, so it is easy to access any contact.

2	1	4	3
---	---	---	---

Unsorted Array

1	2	3	4
---	---	---	---

Array sorted in ascending order

4	3	2	1
---	---	---	---

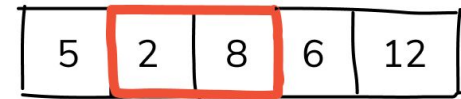
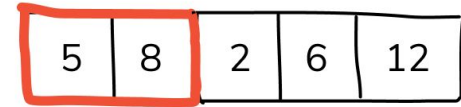
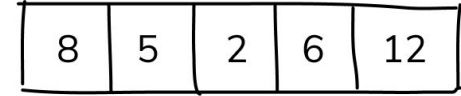
Array sorted in descending order

Data Structures

Bubble Sort

Bubble Sort

- Bubble Sort is the **simplest** sorting algorithm
- Works by **repeatedly swapping** the adjacent elements if they are in the wrong order.
- Each element of the array move to the end in each iteration. Therefore, it is called a bubble sort (arger elements "bubble" up to the top of the list).
- Not suitable for large data sets as its complexity is quite high.



Data Structures

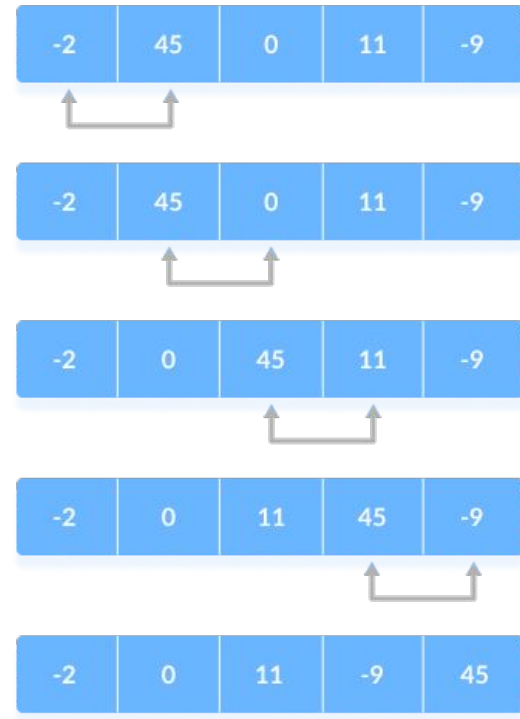
Bubble Sort

Bubble Sort

- Bubble sort goes through several iterations.

First Iteration (Compare and Swap)

- Starting by comparing 1st and 2nd elements.
- If 1st element > 2nd element → they are swapped.
- Then, compare 2nd and 3rd elements. Swap them if they are not in order.
- The above process repeated until the last element.



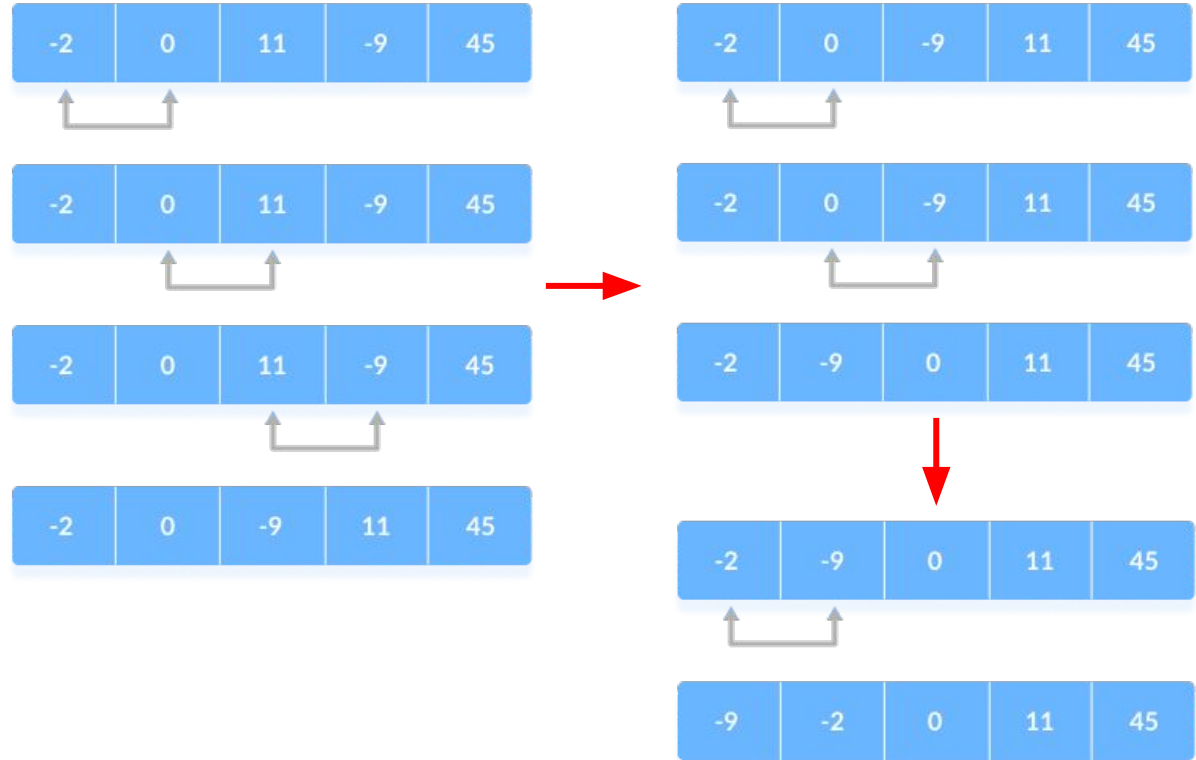
Data Structures

Bubble Sort

Bubble Sort

Remaining Iteration

- The same process of the first iteration repeated for the remaining iterations.
- After each iteration, the largest element among the unsorted elements is placed at the end.



Data Structures

Bubble Sort

Bubble Sort Algorithm

```
For ( $0 \leq i < n-1$ ):  
    For ( $0 \leq j < n-i-1$ ):  
        If (array[j] > array[j+1]):  
            swap these adjacent elements  
        else:  
            continue
```

Bubble Sort Complexity: Worst case: $O(n^2)$

sort in ascending order and the array is in descending order

1st 2nd 3rd last

$$(n-1) + (n-2) + (n-3) + \dots + 1 = n(n-1)/2 \approx n^2$$

“bubble sort requires two loops”

Best case: $O(n)$ If the array is already sorted

Input: array[] = {5, 1, 4, 2, 8}

- (**5** 1 4 2 8) $\rightarrow$ (**1** 5 4 2 8) swaps since $5 > 1$.
- (1 **5** 4 2 8) $\rightarrow$ (1 **4** 5 2 8), Swap since $5 > 4$
- (1 4 **5** 2 8) $\rightarrow$ (1 4 **2** 5 8), Swap since $5 > 2$
- (1 4 2 **5** 8) $\rightarrow$ (1 4 2 **5** 8), already in order ($8 > 5$)

-
- (**1** 4 2 5 8) $\rightarrow$ (1 4 2 5 8) already in order ($4 > 1$)
 - (1 4 2 **5** 8) $\rightarrow$ (1 **2** 4 5 8), Swap since $4 > 2$

-
- (**1** 2 4 5 8) $\rightarrow$ (1 2 4 5 8) already in order ($2 > 1$)

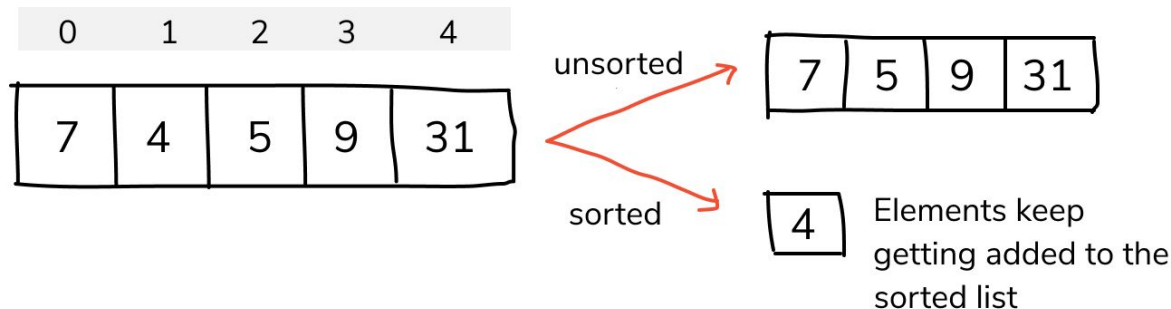


Data Structures

Bubble Sort

Selection sort

- Works by repeatedly finding the **minimum element** (considering ascending order) from the unsorted part and **putting it at the beginning**.
- The algorithm uses two subarrays in a given array:
 - The subarray which is sorted.
 - The remaining subarray was unsorted.
- In every iteration, the minimum element from the unsorted subarray is moved to the sorted subarray.

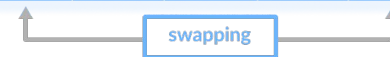
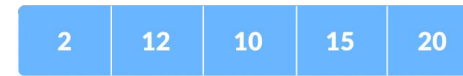
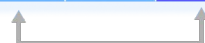
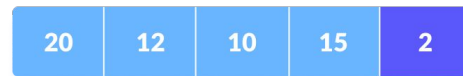
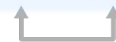
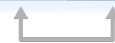
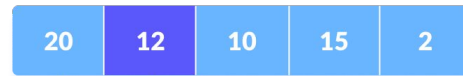


Data Structures

Bubble Sort

Selection Sort

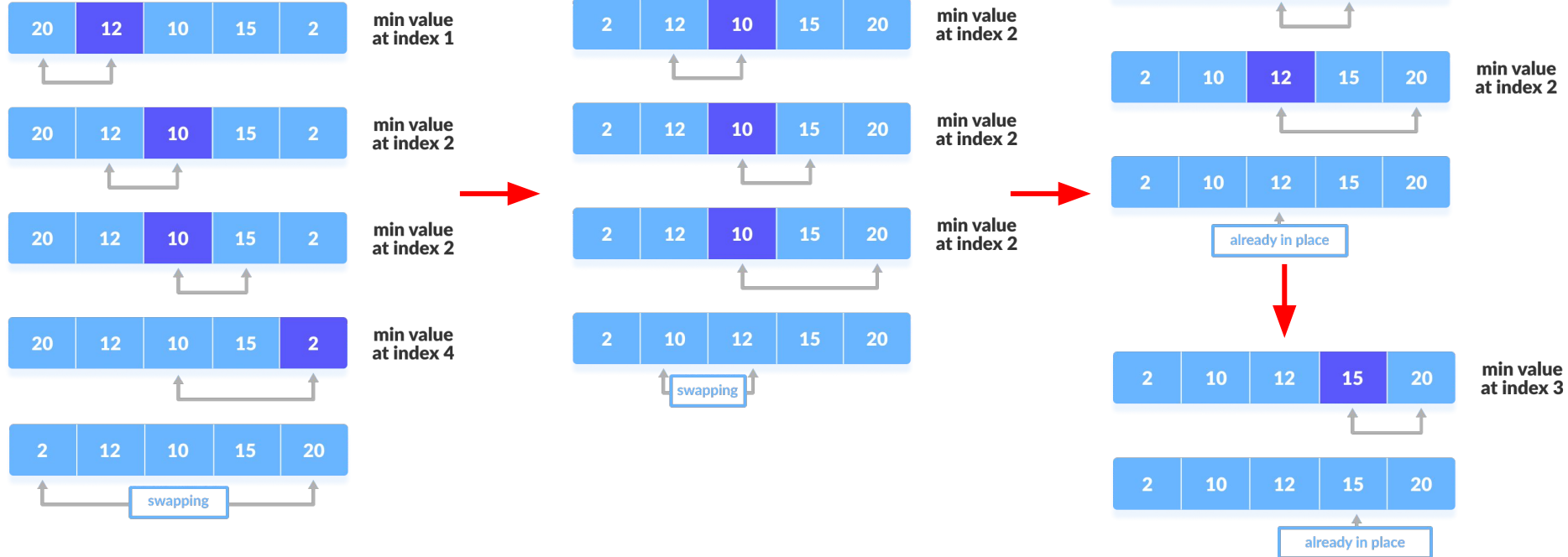
1. Set the first element as minimum.
2. Compare minimum with the 2nd element.
If 2nd element > minimum $\rightarrow$ minimum = 2nd element.
3. The process continues until the last element.
4. After each iteration, minimum is placed in the front of the unsorted list.



Data Structures

Bubble Sort

Selection Sort



Data Structures

Bubble Sort

```
For i = 1 to n-1
  Set min = arr[i]
  Set position = i
  For j = i+1 to n-1 repeat:
    if (min > arr[j])
      Set min = arr[j]
      Set position = j
  swap a[i] with arr[position]
```

Bubble Sort Complexity: Worst case: $O(n^2)$
sort in ascending order and the array is in descending order

1st 2nd 3rd last

$$(n-1) + (n-2) + (n-3) + \dots + 1 = n(n-1)/2 \approx n^2$$

“bubble sort requires two loops”

Input: array[] = {64, 25, 12, 22, 11}

- (**64** 25 12 22 11)
- (**11** 25 12 22 64) *min: 11*
- (**11** 12 25 22 64) *min: 12*
- (**11** 12 22 25 64) *min: 22*
- (**11** 12 22 25 64) *min: 25*
- (**11** 12 22 25 64) *min: 64*

Best case: $O(n)$ If the array is already sorted



Data Structures

Bubble Sort

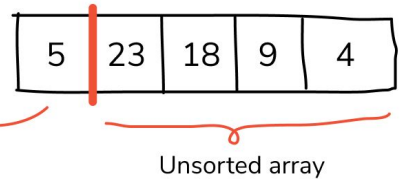
Insertion sort

- The array is split into a sorted and an unsorted part.
- Values from the unsorted part are placed at the correct position in the sorted part.
- It is simple, efficient for small and partially sorted data sets.

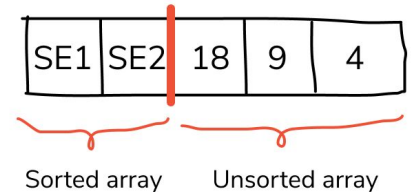
- ❏ *A sub-list is considered as sorted.*
- ❏ *An element which is to be 'inserted' in this sorted sub-list, has to find its appropriate place and then inserted there.*

Hence the name, insertion sort.

Initially, only the 1st element is considered to be a part of the sorted array.



SE ---> Smallest Element
The sorted array continues to grow.



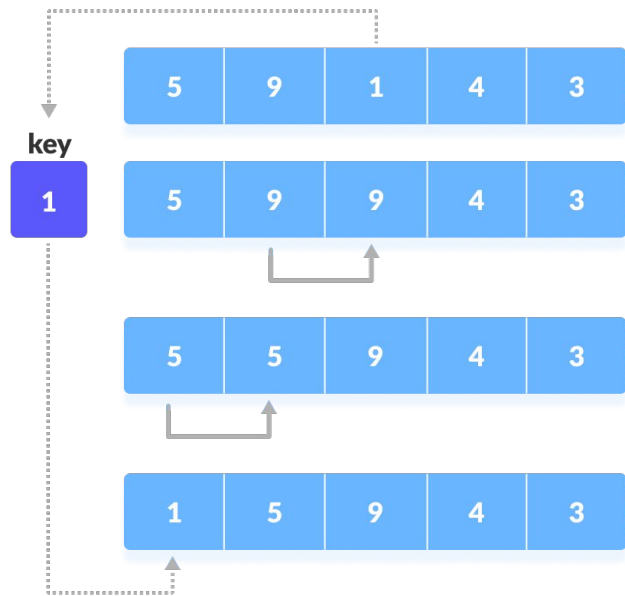
Data Structures

Bubble Sort

Insertion Sort

1. Let the first element to be already sorted.
2. Repeat until list is sorted
 - a. Pick next element
 - b. Compare with all elements in the sorted sub-list
 - c. Shift all the elements in the sorted sub-list that is greater than the value to be sorted
 - d. Insert the value

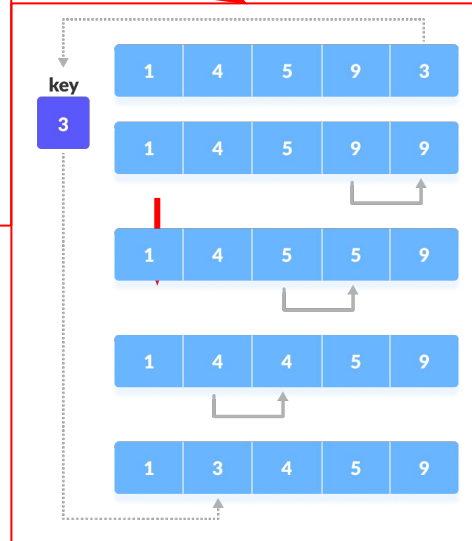
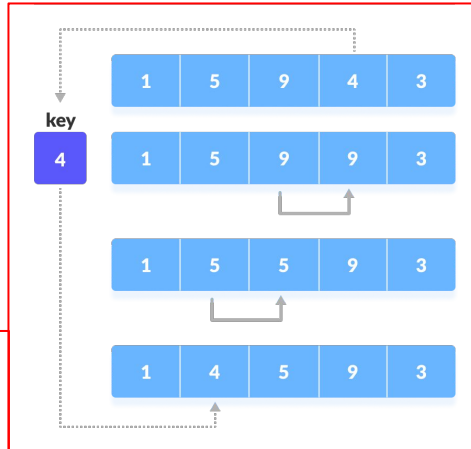
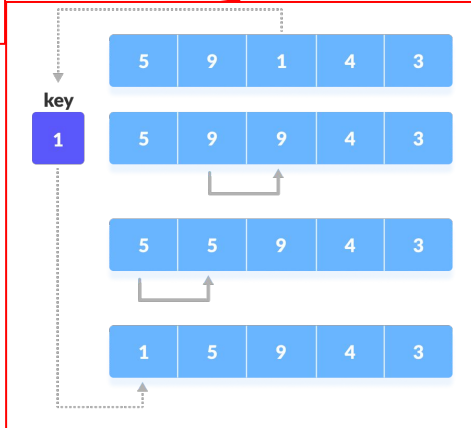
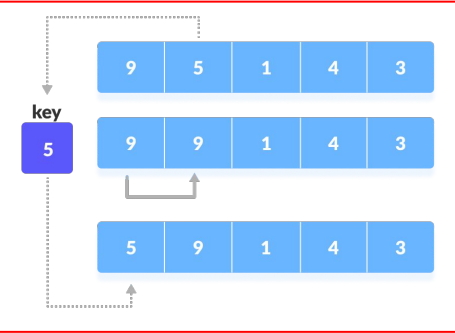
For Example:



Data Structures

Bubble Sort

Insertion Sort



Data Structures

Bubble Sort

For item_x = array[1] to array[n]:

 If item_x < element before:

 Loop to compare item_x to all elements before:

 Move the greater elements one position

 Place item_x to be before all the greater elements

Bubble Sort Complexity: $O(n^2)$

Input: array[] = {4, 3, 2, 10, 12, 1, 5, 6}

